

How To Understand C++20 Coroutines from the Ground Up

C++ Education



VINNIE

FEB 01, 2026



10



2



Share

Introduction

For over two decades, C++ programmers have wrestled with a fundamental challenge: how to write code that waits for things to happen without blocking everything else. Network requests need to complete. Files need to be read. User input must arrive. The traditional solutions—threads, callbacks, and state machines—each carry their own burden of complexity. Threads consume system resources and require careful synchronization. Callbacks scatter your logic across multiple functions. State machines bury simple ideas beneath layers of bookkeeping.

C++20 introduces *coroutines*, a language feature that addresses this challenge directly. A coroutine is a function that can suspend its execution midway through, preserve its state, and resume later from exactly where it left off. This capability transforms the

way you write asynchronous code, allowing you to express complex sequences of operations as straightforward, linear logic.

In this tutorial, you will explore C++20 coroutines from the most basic concepts to practical implementations. You will begin by understanding the problem coroutines solve, then build your first coroutine step by step. By the end, you will have constructed a working generator type and understand the machinery that makes coroutines possible.

Prerequisites

Before beginning this tutorial, you should have the following:

- A C++ compiler with C++20 support (GCC 10+, Clang 14+, or MSVC 2019 16.8+)
- Familiarity with basic C++ concepts: functions, classes, templates, and lambdas
- Understanding of how function calls work: the call stack, local variables, and return values
- A text editor or IDE configured for C++ development

The examples in this tutorial use standard C++20 features. If using GCC, compile with:

```
g++ -std=c++20 -fcoroutines your_file.cpp
```

If using Clang, compile with:

```
clang++ -std=c++20 your_file.cpp
```

If using MSVC, enable C++20 in your project settings or compile with:

```
cl /std:c++20 your_file.cpp
```

Step 1 — Understanding the Problem Coroutines Solve

Before diving into coroutines, you must understand why they exist. Consider a server application that needs to handle an incoming network request. The server must read the request from the network, parse it, possibly read from a database, compute a response, and send that response back. Each of these steps might take time to complete.

In traditional synchronous code, you might write something like this:

```
void handle_request(connection& conn)
{
    std::string request = conn.read();           // blocks until data arrives
    auto parsed = parse_request(request);
    auto data = database.query(parsed.id);       // blocks until database responds
    auto response = compute_response(data);
    conn.write(response);                       // blocks until write completes
}
```

This code reads naturally from top to bottom. The logic flows in a straight line. But there is a problem: while waiting for the network or database, this function blocks the entire thread. If you have thousands of concurrent connections, you would need thousands of threads, each consuming memory and requiring the operating system to schedule them.

The traditional alternative uses callbacks:

```
void handle_request(connection& conn)
{
    conn.async_read([&conn](std::string request) {
        auto parsed = parse_request(request);
        database.async_query(parsed.id, [&conn](auto data) {
            auto response = compute_response(data);
        });
    });
}
```

```

        conn.async_write(response, [&conn]() {
            // request complete
        });
    });
});
}

```

This code does not block. Each operation starts, registers a callback, and returns immediately. When the operation completes, the callback runs. But look what has happened to the code: three levels of nesting, logic scattered across multiple lambda functions, and local variables that cannot be shared between callbacks without careful lifetime management.

David Mazières, in his exploration of C++ coroutines, described the pain of this approach vividly. In his SMTP server code, a single logical operation named `cmd_rcpt` had to be split across seven separate functions: `cmd_rcpt`, `cmd_rcpt_0`, `cmd_rcpt_2`, `cmd_rcpt_3`, `cmd_rcpt_4`, `cmd_rcpt_5`, and `cmd_rcpt_6`. Each function represented a different return point from an asynchronous operation. The logic of a single command was scattered across the codebase.

Coroutines solve this problem by allowing you to write code that looks synchronous but behaves asynchronously:

```

task<void> handle_request(connection& conn)
{
    std::string request = co_await conn.async_read();
    auto parsed = parse_request(request);
    auto data = co_await database.async_query(parsed.id);
    auto response = compute_response(data);
    co_await conn.async_write(response);
}

```

This code reads just like the original blocking version. The logic flows from top to bottom. Local variables like `request`, `parsed`, and `data` exist naturally in their scope. Yet the function suspends at each `co_await` point, allowing other work to proceed while waiting.

The variable `request` maintains its value even though the function may suspend and resume multiple times. This is the fundamental capability that coroutines provide: the preservation of local state across suspension points.

You have now seen the problem that coroutines solve. The callback approach fragments your logic. Coroutines restore the natural flow of code while maintaining asynchronous behavior.

Step 2 — Recognizing Coroutines by Their Keywords

A coroutine in C++20 looks almost like a regular function. The difference lies in what appears inside the function body. A function becomes a coroutine when it contains any of three special keywords: `co_await`, `co_yield`, or `co_return`.

The keyword `co_await` suspends the coroutine and waits for some operation to complete. When you write `co_await expr`, the coroutine saves its state, pauses execution, and potentially allows other code to run. When the awaited operation completes, the coroutine resumes from exactly where it left off.

The keyword `co_yield` produces a value and suspends the coroutine. This is useful for generators—functions that produce a sequence of values one at a time. After yielding a value, the coroutine pauses until someone asks for the next value.

The keyword `co_return` completes the coroutine and optionally provides a final result. Unlike a regular `return` statement, `co_return` interacts with the coroutine machinery to properly finalize the coroutine's state.

Here is the simplest possible coroutine:

```
#include <coroutine>

struct SimpleCoroutine {
    struct promise_type {
        SimpleCoroutine get_return_object() { return {}; }
        std::suspend_never initial_suspend() { return {}; }
        std::suspend_never final_suspend() noexcept { return {}; }
        void return_void() {}
        void unhandled_exception() {}
    };
};

SimpleCoroutine my_first_coroutine()
{
    co_return; // This makes it a coroutine
}
```

Do not worry about the `promise_type` structure yet. You will explore it in detail later. For now, observe that the presence of `co_return` transforms what looks like a regular function into a coroutine.

If you try to compile a function with these keywords but without proper infrastructure, the compiler will produce errors. The C++ coroutine mechanism requires certain types

and functions to exist. This is why the example includes the `promise_type` nested structure—it provides the minimum scaffolding the compiler needs.

The distinction between regular functions and coroutines matters because they behave fundamentally differently at runtime:

- A regular function allocates its local variables on the stack. When it returns, those variables are gone.
- A coroutine allocates its local variables in a heap-allocated *coroutine frame*. When it suspends, those variables persist. When it resumes, they are still there.

This persistence of state is what allows coroutines to pause and resume while maintaining their local variables.

You have now learned to recognize coroutines by their keywords. The presence of `co_await`, `co_yield`, or `co_return` signals that a function is a coroutine with special runtime behavior.

Step 3 — Understanding Suspension and Resumption

The heart of coroutines is the ability to suspend execution and resume it later. To understand how this works, you must examine what happens when a coroutine

suspends.

When you call a regular function, the system allocates space on the call stack for the function’s local variables and parameters. When the function returns, this stack space is reclaimed. The function’s state exists only during the call.

When you call a coroutine, something different happens. The system allocates a *coroutine frame* on the heap. This frame holds the coroutine’s local variables, parameters, and information about where execution should resume. Because the frame lives on the heap rather than the stack, it persists even when the coroutine is not actively running.

Consider this example:

```
#include <coroutine>
#include <iostream>

struct ReturnObject {
    struct promise_type {
        ReturnObject get_return_object() { return {}; }
        std::suspend_never initial_suspend() { return {}; }
        std::suspend_never final_suspend() noexcept { return {}; }
        void return_void() {}
        void unhandled_exception() {}
    }
};
```

```

    };

};

struct Awaiter {
    std::coroutine_handle<>* handle_out;

    bool await_ready() { return false; }
    void await_suspend(std::coroutine_handle<> h) {
        *handle_out = h;
    }
    void await_resume() {}
};

ReturnObject counter(std::coroutine_handle<>* handle)
{
    Awaiter awaiter{handle};

    for (unsigned i = 0; ; ++i) {
        std::cout << "counter: " << i << std::endl;
        co_await awaiter;
    }
}

int main()
{

```

```

    std::coroutine_handle<> h;
    counter(&h);

    for (int i = 0; i < 3; ++i) {
        std::cout << "main: resuming" << std::endl;
        h();
    }

    h.destroy();
}

```

Output:

```

counter: 0
main: resuming
counter: 1
main: resuming
counter: 2
main: resuming
counter: 3

```

Study what happens in this example:

1. The `main` function calls `counter`, passing the address of a coroutine handle.
2. The `counter` coroutine begins executing. It prints “counter: 0” and then reaches `co_await awaiter`.
3. The `co_await` expression checks if the awaiter is ready by calling `await_ready()`. It returns `false`, so suspension proceeds.
4. The coroutine saves its state—including the value of `i`—to the coroutine frame.
5. The `await_suspend` method receives a handle to the suspended coroutine and stores it in `main`’s variable `h`.
6. Control returns to `main`, which now holds a handle to the suspended coroutine.
7. The `main` function calls `h()`, which resumes the coroutine.
8. The coroutine continues from where it left off, increments `i`, prints its new value, and suspends again.
9. This cycle repeats until `main` destroys the coroutine.

The variable `i` inside `counter` maintains its value across all these suspension and resumption cycles. It starts at 0, increments to 1, then 2, then 3. Each time the coroutine resumes, `i` is exactly where it was when the coroutine suspended.

A `std::coroutine_handle<>` is a lightweight object, similar to a pointer. It references the coroutine frame on the heap. Calling the handle (using `h()` or `h.resume()`) resumes the coroutine. The handle does not own the coroutine frame—you must eventually call `h.destroy()` to free the memory.

The Awaiter type in this example demonstrates the three methods that `co_await` uses:

- `await_ready()`: Returns `true` if the result is immediately available and no suspension is needed. Returns `false` to proceed with suspension.
- `await_suspend(handle)`: Called when the coroutine suspends. Receives the coroutine handle, allowing external code to later resume the coroutine.
- `await_resume()`: Called when the coroutine resumes. Its return value becomes the value of the `co_await` expression.

The C++ standard library provides two predefined awaiters: `std::suspend_always` and `std::suspend_never`. As their names suggest, `suspend_always::await_ready()` always returns `false` (always suspend), while `suspend_never::await_ready()` always returns `true` (never suspend).

You have now seen how suspension and resumption work. The coroutine frame preserves state on the heap, and the coroutine handle provides a way to resume

execution.

Step 4 — Understanding the Promise Type

Every coroutine has an associated *promise type*. This type acts as a controller for the coroutine, defining how it behaves at key points in its lifecycle. The promise type is not something you pass to the coroutine—it is a nested type inside the coroutine's return type that the compiler uses automatically.

The compiler expects to find a type named `promise_type` nested inside your coroutine's return type. If your coroutine returns `Generator<int>`, the compiler looks for `Generator<int>::promise_type`. This promise type must provide certain methods that the compiler calls at specific points during the coroutine's execution.

Here are the required methods:

`get_return_object()`: Called to create the object that will be returned to the caller of the coroutine. This happens before the coroutine body begins executing.

`initial_suspend()`: Called immediately after `get_return_object()`. Returns an awaiter that determines whether the coroutine should suspend before running any

of its body. Return `std::suspend_never{}` to start executing immediately, or `std::suspend_always{}` to suspend before the first statement.

`final_suspend()`: Called when the coroutine completes (either normally or via exception). Returns an awaiter that determines whether to suspend one last time or destroy the coroutine state immediately. This method must be `noexcept`.

`return_void()` or `return_value(v)`: Called when the coroutine executes `co_return` or falls off the end of its body. Use `return_void()` if the coroutine does not return a value; use `return_value(v)` if it does. You must provide exactly one of these, matching how your coroutine returns.

`unhandled_exception()`: Called if an exception escapes the coroutine body. Typically you either rethrow the exception, store it for later, or terminate the program.

The compiler transforms your coroutine body into something resembling this pseudocode:

```
{
    promise_type promise;
    auto return_object = promise.get_return_object();

    co_await promise.initial_suspend();
```



```

    try {
        // your coroutine body goes here
    }
    catch (...) {
        promise.unhandled_exception();
    }

    co_await promise.final_suspend();
}
// coroutine frame is destroyed when control flows off the end

```

This transformation reveals important details. The return object is created before `initial_suspend()` runs, so it is available even if the coroutine suspends immediately. The `final_suspend()` determines whether the coroutine frame persists after completion—if it returns `suspend_always`, you must manually destroy the coroutine; if it returns `suspend_never`, the frame is destroyed automatically.

Consider this example that demonstrates promise type behavior:

```

#include <coroutine>
#include <iostream>

```

```

struct TracePromise {
    struct promise_type {
        promise_type() {
            std::cout << "promise constructed" << std::endl;
        }
        ~promise_type() {
            std::cout << "promise destroyed" << std::endl;
        }

        TracePromise get_return_object() {
            std::cout << "get_return_object called" << std::endl;
            return {};
        }

        std::suspend_never initial_suspend() {
            std::cout << "initial_suspend called" << std::endl;
            return {};
        }

        std::suspend_always final_suspend() noexcept {
            std::cout << "final_suspend called" << std::endl;
            return {};
        }

        void return_void() {
            std::cout << "return_void called" << std::endl;
        }

        void unhandled_exception() {

```

```

        std::cout << "unhandled_exception called" << std::endl;
    }
};

std::coroutine_handle<promise_type> handle;
};

TracePromise trace_coroutine()
{
    std::cout << "coroutine body begins" << std::endl;
    co_return;
}

int main()
{
    std::cout << "calling coroutine" << std::endl;
    auto result = trace_coroutine();
    std::cout << "coroutine returned" << std::endl;
}

```

Output:

```

calling coroutine
promise constructed

```

```

get_return_object called
initial_suspend called
coroutine body begins
return_void called
final_suspend called
coroutine returned

```

Notice that the promise is constructed first, then `get_return_object()` creates the return value, then `initial_suspend()` runs. Since `initial_suspend()` returns `suspend_never`, the coroutine body executes immediately. After `co_return`, `return_void()` is called, followed by `final_suspend()`. Since `final_suspend()` returns `suspend_always`, the coroutine suspends one last time, and the promise is not destroyed until the coroutine handle is explicitly destroyed.

One important warning: if your coroutine can fall off the end of its body without executing `co_return`, and your promise type lacks a `return_void()` method, the behavior is undefined. This is a dangerous pitfall. Always ensure your promise type has `return_void()` if there is any code path that might reach the end of the coroutine body without an explicit `co_return`.

You have now learned how the promise type controls coroutine behavior. The methods on the promise type let you customize initialization, suspension, value delivery, and

cleanup.

Step 5 — Building a Generator with `co_yield`

One of the most common uses for coroutines is building *generators*—functions that produce a sequence of values on demand. Instead of computing all values upfront and storing them in a container, a generator computes each value when requested.

The `co_yield` keyword makes this pattern elegant. When a coroutine executes `co_yield value`, it delivers the value to its caller and suspends. The next time the coroutine resumes, it continues from just after the `co_yield`.

Here is how `co_yield` works internally. The expression `co_yield value` is transformed by the compiler into:

```
co_await promise.yield_value(value)
```

The `yield_value` method is a new method you must add to your promise type. It receives the yielded value, typically stores it somewhere accessible, and returns an awaiter (usually `std::suspend_always`) to suspend the coroutine.

Here is a complete generator example:

```
#include <coroutine>
#include <iostream>

struct Generator {
    struct promise_type {
        int current_value;

        Generator get_return_object() {
            return Generator{
                std::coroutine_handle<promise_type>::from_promise(*this)
            };
        }
        std::suspend_always initial_suspend() { return {}; }
        std::suspend_always final_suspend() noexcept { return {}; }
        std::suspend_always yield_value(int value) {
            current_value = value;
            return {};
        }
        void return_void() {}
        void unhandled_exception() { std::terminate(); }
    };

    std::coroutine_handle<promise_type> handle;

    Generator(std::coroutine_handle<promise_type> h) : handle(h) {}
};
```

```

~Generator() { if (handle) handle.destroy(); }

// Disable copying
Generator(const Generator&) = delete;
Generator& operator=(const Generator&) = delete;

// Enable moving
Generator(Generator&& other) noexcept
    : handle(other.handle) { other.handle = nullptr; }
Generator& operator=(Generator&& other) noexcept {
    if (this != &other) {
        if (handle) handle.destroy();
        handle = other.handle;
        other.handle = nullptr;
    }
    return *this;
}

bool next() {
    if (!handle || handle.done())
        return false;
    handle.resume();
    return !handle.done();
}

```

```

int value() const {
    return handle.promise().current_value;
}

};

Generator count_to(int n)
{
    for (int i = 1; i <= n; ++i) {
        co_yield i;
    }
}

int main()
{
    auto gen = count_to(5);

    while (gen.next()) {
        std::cout << gen.value() << std::endl;
    }
}

```

Output:

```
1
2
3
4
5
```

Study the key parts of this example:

The `yield_value` method stores the yielded value in `current_value` and returns `suspend_always` to pause the coroutine after each yield.

The `initial_suspend` returns `suspend_always`, which means the coroutine suspends before executing any of its body. This is important—it means the first call to `next()` is what starts the coroutine running.

The `get_return_object` method creates the Generator object and stores a handle to the coroutine. Notice the expression `std::coroutine_handle<promise_type>::from_promise(*this)`. This static method creates a coroutine handle from a reference to the promise object. Since the promise object lives inside the coroutine frame at a known offset, this conversion is possible.

The Generator class manages the coroutine handle's lifetime. The destructor calls `handle.destroy()` to free the coroutine frame. The class disables copying (copying handles would be problematic) but enables moving.

The `next()` method resumes the coroutine and returns `true` if the coroutine produced a value, or `false` if the coroutine has completed. The `value()` method retrieves the most recently yielded value from the promise.

Here is a more interesting generator that produces the Fibonacci sequence:

```
Generator fibonacci()
{
    int a = 0, b = 1;
    while (true) {
        co_yield a;
        int next = a + b;
        a = b;
        b = next;
    }
}

int main()
{
    auto fib = fibonacci();
```

```

    for (int i = 0; i < 10 && fib.next(); ++i) {
        std::cout << fib.value() << " ";
    }
    std::cout << std::endl;
}

```

Output:

```
0 1 1 2 3 5 8 13 21 34
```

The Fibonacci generator runs an infinite loop internally. It will produce values forever. But because it yields and suspends after each value, the caller controls when (and whether) to ask for more values. The generator only computes values on demand.

This is the power of generators. The variables `a` and `b` persist across yields because they live in the coroutine frame on the heap. Each call to `next()` resumes the coroutine, which computes the next Fibonacci number, yields it, and suspends again.

You have now built a working generator using `co_yield`. The promise type's `yield_value` method receives yielded values, and the `Generator` class provides an interface for retrieving them.

Step 6 — Understanding Return Objects and Coroutine Handles

You have seen coroutine handles and return objects in previous examples. Now you will examine them more closely to understand their relationship and how information flows between them.

A *coroutine handle* (`std::coroutine_handle<>`) is a lightweight object that refers to a suspended coroutine. It is similar to a pointer: it does not own the memory it references, and copying it does not copy the coroutine. You can resume the coroutine by calling the handle (using `handle()` or `handle.resume()`), query whether the coroutine has completed with `handle.done()`, and destroy the coroutine frame with `handle.destroy()`.

The coroutine handle is a template. `std::coroutine_handle<>` (equivalent to `std::coroutine_handle<void>`) is the most basic form—it can reference any coroutine but provides no access to the promise object.

`std::coroutine_handle<PromiseType>` is a more specific form that knows about a particular promise type. This typed handle can be converted to the void handle, and it provides a `promise()` method that returns a reference to the promise object.

The *return object* is what the caller receives when calling a coroutine. It is the type that appears in the coroutine's declaration. When you write:

```
Generator my_coroutine() {  
    co_yield 42;  
}
```

The return type is `Generator`, and when you call `my_coroutine()`, you receive a `Generator` object.

The return object is created by calling `promise.get_return_object()` before the coroutine body begins. This happens early in the coroutine's lifecycle, giving the return object a chance to capture the coroutine handle. Here is the sequence:

1. The coroutine frame is allocated on the heap.
2. The promise object is constructed inside the frame.
3. `promise.get_return_object()` is called, creating the return object.
4. `co_await promise.initial_suspend()` executes.
5. The coroutine body begins (if `initial_suspend` did not suspend).
6. The return object is given to the caller.

The key insight is that `get_return_object()` runs before `initial_suspend()`. This means:

- If `initial_suspend()` returns `suspend_always`, the coroutine suspends before any user code runs, but the return object already exists and contains the coroutine handle.
- If `initial_suspend()` returns `suspend_never`, the coroutine runs immediately, and the return object is still created first.

Inside `get_return_object()`, you can obtain the coroutine handle using the static method `coroutine_handle::from_promise(*this)`. Since `get_return_object()` is called on the promise object (as `this`), this method returns a handle to the coroutine containing that promise.

Here is an example that demonstrates the relationship:

```
#include <coroutine>  
#include <iostream>  
  
struct Task {  
    struct promise_type {  
        Task get_return_object() {  
            std::cout << "Creating return object" << std::endl;  

```

```

        return Task{
            std::coroutine_handle<promise_type>::from_promise(*this)
        };
    }
    std::suspend_always initial_suspend() {
        std::cout << "Initial suspend" << std::endl;
        return {};
    }
    std::suspend_always final_suspend() noexcept {
        std::cout << "Final suspend" << std::endl;
        return {};
    }
    void return_void() {}
    void unhandled_exception() {}
};

std::coroutine_handle<promise_type> handle;

Task(std::coroutine_handle<promise_type> h) : handle(h) {}
~Task() { if (handle) handle.destroy(); }

Task(Task&& other) noexcept : handle(other.handle) {
    other.handle = nullptr;
}

```

```

void resume() { handle.resume(); }
bool done() const { return handle.done(); }
};

Task example_task()
{
    std::cout << "Task body: part 1" << std::endl;
    co_await std::suspend_always{};
    std::cout << "Task body: part 2" << std::endl;
}

int main()
{
    std::cout << "Before calling coroutine" << std::endl;

    Task task = example_task();

    std::cout << "After calling coroutine, before first resume" << std::endl;
    task.resume();

    std::cout << "After first resume, before second resume" << std::endl;
    task.resume();

    std::cout << "After second resume" << std::endl;
}

```


Output:

```
Before calling coroutine
Creating return object
Initial suspend
After calling coroutine, before first resume
Task body: part 1
After first resume, before second resume
Task body: part 2
Final suspend
After second resume
```

Follow the execution flow:

1. Before `example_task()` is called, nothing has happened.
2. Calling `example_task()` creates the coroutine frame, constructs the promise, and calls `get_return_object()`.
3. The return object (Task) is created with a handle to the coroutine.
4. `initial_suspend()` runs and returns `suspend_always`, so the coroutine suspends immediately.
5. Control returns to `main`, which now holds the Task object.
6. The first `resume()` runs “Task body: part 1”, then hits `co_await suspend_always{}` and suspends.
7. The second `resume()` runs “Task body: part 2”, then falls off the end, triggering `final_suspend()`.
8. Since `final_suspend()` returns `suspend_always`, the coroutine suspends one final time.
9. When Task’s destructor runs (at the end of `main`), it destroys the coroutine handle.

The return object provides an interface to the caller. It hides the details of coroutine handles and promises behind whatever API makes sense for your use case. For a generator, the return object provides methods like `next()` and `value()`. For a task, it might provide `resume()` and `done()`. The return object owns the coroutine handle and is responsible for destroying it.

You have now seen how return objects and coroutine handles work together. The return object is the caller’s view of the coroutine, while the handle is the mechanism for resuming and managing the coroutine’s lifetime.

Step 7 — Completing Coroutines with `co_return`

You have seen coroutines that yield sequences of values and suspend indefinitely. Now you will learn how coroutines complete their execution using `co_return`.

A coroutine completes in one of three ways:

1. It executes `co_return;` (returning void)
2. It executes `co_return expression;` (returning a value)
3. Execution falls off the end of the coroutine body

For case 1 and 3, the compiler calls `promise.return_void()`. For case 2, the compiler calls `promise.return_value(expression)`. You must provide exactly one of these methods in your promise type, matching how your coroutine returns.

When a coroutine completes (by any of these means), it then executes `co_await promise.final_suspend()`. The awaiter returned by `final_suspend()` determines what happens next:

- If it suspends (like `suspend_always`), the coroutine frame remains valid. The caller can still access the promise object and must eventually call `handle.destroy()` to free the memory.
- If it does not suspend (like `suspend_never`), the coroutine frame is destroyed automatically. Any handles to the coroutine become dangling pointers.

The choice between these behaviors matters. If your caller needs to access the result stored in the promise after the coroutine completes, use `suspend_always`. If the coroutine's completion signals some external mechanism (like releasing a semaphore) and the result is not needed, you might use `suspend_never` to avoid manual cleanup.

Here is an example of a coroutine that returns a value:

```
#include <coroutine>
#include <iostream>
#include <optional>

struct ComputeResult {
    struct promise_type {
        std::optional<int> result;

        ComputeResult get_return_object() {
            return ComputeResult{
                std::coroutine_handle<promise_type>::from_promise(*this)
            };
        }

        std::suspend_always initial_suspend() { return {}; }
        std::suspend_always final_suspend() noexcept { return {}; }
        void return_value(int value) {
            result = value;
        }
    };
};
```

```

    }
    void unhandled_exception() {
        result = std::nullopt;
    }
};

std::coroutine_handle<promise_type> handle;

ComputeResult(std::coroutine_handle<promise_type> h) : handle(h) {}
~ComputeResult() { if (handle) handle.destroy(); }

ComputeResult(ComputeResult&& other) noexcept : handle(other.handle) {
    other.handle = nullptr;
}

void run() {
    while (!handle.done()) {
        handle.resume();
    }
}

std::optional<int> get_result() const {
    return handle.promise().result;
}
};

```

```

ComputeResult compute_sum(int n)
{
    int sum = 0;
    for (int i = 1; i <= n; ++i) {
        sum += i;
        co_await std::suspend_always{}; // yield control periodically
    }
    co_return sum;
}

int main()
{
    auto computation = compute_sum(5);
    computation.run();

    if (auto result = computation.get_result()) {
        std::cout << "Result: " << *result << std::endl;
    }
}

```

Output:

Result: 15

The `compute_sum` coroutine adds numbers from 1 to `n`, periodically yielding control with `co_await suspend_always{}`. When the loop completes, it executes `co_return sum`, which calls `promise.return_value(sum)`, storing the result in the promise.

Because `final_suspend()` returns `suspend_always`, the coroutine frame remains valid after completion. The `get_result()` method can access `handle.promise().result` to retrieve the computed value.

You can query whether a coroutine has completed using `handle.done()`. This method returns `true` after the coroutine has executed `co_return` (or fallen off the end) and completed the `final_suspend` awaiter. Do not confuse `handle.done()` with `handle.operator bool()`. The boolean conversion only checks if the handle is non-null; it does not indicate completion.

A critical warning about undefined behavior: if your coroutine can fall off the end of its body and your promise type does not have a `return_void()` method, the behavior is undefined. This is dangerous because the compiler may not warn you.

Always ensure your promise type has `return_void()` if any code path might reach the end of the coroutine without an explicit `co_return`.

Here is the same computation rewritten to fall off the end instead of using explicit `co_return`:

```
struct ComputeResult2 {
    struct promise_type {
        int result = 0;

        ComputeResult2 get_return_object() {
            return ComputeResult2{
                std::coroutine_handle<promise_type>::from_promise(*this)
            };
        }
        std::suspend_always initial_suspend() { return {}; }
        std::suspend_always final_suspend() noexcept { return {}; }
        void return_void() {} // Required because we fall off the end
        void unhandled_exception() {}
    };

    std::coroutine_handle<promise_type> handle;
    // ... rest of the class
};
```

```

ComputeResult2 compute_sum2(int n)
{
    auto& result = co_await GetPromiseAwaiter{}; // hypothetical
    int sum = 0;
    for (int i = 1; i <= n; ++i) {
        sum += i;
        co_await std::suspend_always{};
    }
    result = sum;
    // Falls off the end - calls promise.return_void()
}

```

In this version, we store the result in the promise before falling off the end. The `return_void()` method must exist even though it does nothing, because the coroutine reaches the end of its body.

You have now learned how coroutines complete execution. The `co_return` statement (or falling off the end) triggers the promise's return methods, and `final_suspend` determines whether the coroutine frame persists.

Step 8 — Building a Generic Generator

You have learned all the pieces needed to build a reusable generator type. In this step, you will assemble them into a template class that works with any value type.

A production-quality generator needs to handle several concerns:

1. Store and retrieve yielded values of any type
2. Manage the coroutine handle's lifetime correctly
3. Propagate exceptions from the coroutine to the caller
4. Provide a clean iteration interface

Here is a complete generic generator:

```

#include <coroutine>
#include <exception>
#include <utility>

template<typename T>
class Generator {
public:
    struct promise_type {
        T value;
        std::exception_ptr exception;
    }
};

```

```

Generator get_return_object() {
    return Generator{Handle::from_promise(*this)};
}

std::suspend_always initial_suspend() noexcept {
    return {};
}

std::suspend_always final_suspend() noexcept {
    return {};
}

std::suspend_always yield_value(T v) {
    value = std::move(v);
    return {};
}

void return_void() noexcept {}

void unhandled_exception() {
    exception = std::current_exception();
}

template<typename U>
std::suspend_never await_transform(U&&) = delete;

```

```

};

using Handle = std::coroutine_handle<promise_type>;

private:
    Handle handle_;

public:
    explicit Generator(Handle h) : handle_(h) {}

    ~Generator() {
        if (handle_) {
            handle_.destroy();
        }
    }

    Generator(const Generator&) = delete;
    Generator& operator=(const Generator&) = delete;

    Generator(Generator&& other) noexcept
        : handle_(std::exchange(other.handle_, nullptr)) {}

    Generator& operator=(Generator&& other) noexcept {
        if (this != &other) {
            if (handle_) {

```

```

        handle_.destroy();
    }
    handle_ = std::exchange(other.handle_, nullptr);
}
return *this;
}

class iterator {
    Handle handle_;

public:
    using iterator_category = std::input_iterator_tag;
    using value_type = T;
    using difference_type = std::ptrdiff_t;
    using pointer = T*;
    using reference = T&;

    iterator() : handle_(nullptr) {}
    explicit iterator(Handle h) : handle_(h) {}

    iterator& operator++() {
        handle_.resume();
        if (handle_.done()) {
            auto& promise = handle_.promise();
            handle_ = nullptr;

```

```

            if (promise.exception) {
                std::rethrow_exception(promise.exception);
            }
        }
        return *this;
    }

    iterator operator++(int) {
        iterator temp = *this;
        ++(*this);
        return temp;
    }

    T& operator*() const {
        return handle_.promise().value;
    }

    T* operator->() const {
        return &handle_.promise().value;
    }

    bool operator==(const iterator& other) const {
        return handle_ == other.handle_;
    }
}

```

```

        bool operator!=(const iterator& other) const {
            return !(*this == other);
        }
};

iterator begin() {
    if (handle_) {
        handle_.resume();
        if (handle_.done()) {
            auto& promise = handle_.promise();
            if (promise.exception()) {
                std::rethrow_exception(promise.exception());
            }
            return iterator{};
        }
    }
    return iterator{handle_};
}

iterator end() {
    return iterator{};
}
};

```

This generator provides a standard iterator interface, allowing use in range-based for loops:

```

Generator<int> range(int start, int end)
{
    for (int i = start; i < end; ++i) {
        co_yield i;
    }
}

Generator<int> squares(int n)
{
    for (int i = 0; i < n; ++i) {
        co_yield i * i;
    }
}

int main()
{
    std::cout << "Range 1 to 5:" << std::endl;
    for (int x : range(1, 6)) {
        std::cout << x << " ";
    }
    std::cout << std::endl;
}

```



```
std::cout << "First 5 squares:" << std::endl;
for (int x : squares(5)) {
    std::cout << x << " ";
}
std::cout << std::endl;
}
```

Output:

```
Range 1 to 5:
1 2 3 4 5
First 5 squares:
0 1 4 9 16
```

Several design choices in this generator deserve explanation:

initial_suspend() returns suspend_always: The coroutine suspends before running any user code. This means `begin()` must resume the coroutine to get the first value. This design prevents work from being done if the generator is never iterated.

final_suspend() returns suspend_always: The coroutine frame persists after completion. This is necessary because the iterator needs to check `handle_.done()`

and potentially access the exception stored in the promise. If `final_suspend()` returned `suspend_never`, the handle would become invalid before these checks could occur.

Exception handling: The `unhandled_exception()` method stores the current exception in the promise using `std::current_exception()`. The iterator's `operator++` and `begin()` check for this exception and rethrow it using `std::rethrow_exception()`. This propagates exceptions from the coroutine to the calling code.

await_transform is deleted: This prevents using `co_await` inside the generator. A generator should only yield values, not await other operations. Deleting `await_transform` makes any use of `co_await` inside a `Generator<T>` coroutine a compile error.

Move semantics: The generator is movable but not copyable. Copying a coroutine handle would create aliasing problems—both copies would refer to the same coroutine frame, and destroying one would invalidate the other. Moving transfers ownership cleanly.

Here is an example demonstrating exception propagation:

```

Generator<int> may_throw(bool should_throw)
{
    co_yield 1;
    co_yield 2;
    if (should_throw) {
        throw std::runtime_error("Generator error");
    }
    co_yield 3;
}

int main()
{
    try {
        for (int x : may_throw(true)) {
            std::cout << x << std::endl;
        }
    }
    catch (const std::exception& e) {
        std::cout << "Caught: " << e.what() << std::endl;
    }
}

```

Output:

```

1
2
Caught: Generator error

```

The exception thrown inside the generator propagates to the calling code and can be caught normally.

You have now built a production-quality generic generator. It handles value types, manages coroutine lifetime, propagates exceptions, and provides a standard iterator interface.

Step 9 — Handling Exceptions in Coroutines

Exceptions in coroutines require special attention. Because a coroutine can suspend and resume across different call stacks, the normal exception propagation mechanism does not work directly. The promise type's `unhandled_exception()` method provides the hook for handling exceptions that escape the coroutine body.

When an exception is thrown inside a coroutine and not caught within the coroutine, the following happens:

1. The exception is caught by the implicit try-catch block surrounding the coroutine body.
2. `promise.unhandled_exception()` is called while the exception is still active.
3. After `unhandled_exception()` returns, `co_await promise.final_suspend()` executes.
4. The coroutine completes (either suspended or destroyed, depending on `final_suspend`).

Inside `unhandled_exception()`, you have several options:

Terminate the program: Call `std::terminate()`. This is the safest option if you cannot handle exceptions.

```
void unhandled_exception() {  
    std::terminate();  
}
```

Store the exception for later: Use `std::current_exception()` to capture the exception and store it in the promise. The caller can later check for the exception and rethrow it.

```
void unhandled_exception() {  
    exception_ = std::current_exception();  
}
```

Rethrow the exception: Call `throw;` to rethrow the exception. This propagates the exception to whoever is currently running the coroutine, but be careful—this may not be the original caller if the coroutine has been resumed from a different context.

```
void unhandled_exception() {  
    throw;  
}
```

Swallow the exception: Do nothing. This silences the exception, which is almost always a mistake but might be appropriate in specific circumstances.

```
void unhandled_exception() {  
    // Exception is silently ignored  
}
```

The stored exception pattern is most useful for generators and tasks where the caller expects to receive results:

```

#include <coroutine>
#include <exception>
#include <iostream>
#include <stdexcept>

struct Task {
    struct promise_type {
        std::exception_ptr exception;

        Task get_return_object() {
            return Task{std::coroutine_handle<promise_type>::from_promise(*this)};
        }
        std::suspend_always initial_suspend() { return {}; }
        std::suspend_always final_suspend() noexcept { return {}; }
        void return_void() {}
        void unhandled_exception() {
            exception = std::current_exception();
        }
    };

    std::coroutine_handle<promise_type> handle;

    Task(std::coroutine_handle<promise_type> h) : handle(h) {}
    ~Task() { if (handle) handle.destroy(); }
};

```

```

void run() {
    handle.resume();
}

void check_exception() {
    if (handle.promise().exception) {
        std::rethrow_exception(handle.promise().exception);
    }
}

Task risky_operation()
{
    std::cout << "Starting risky operation" << std::endl;
    throw std::runtime_error("Something went wrong");
    co_return; // Never reached
}

int main()
{
    Task task = risky_operation();

    try {
        task.run();
        task.check_exception();
    }
}

```

```

        std::cout << "Operation completed successfully" << std::endl;
    }
    catch (const std::exception& e) {
        std::cout << "Operation failed: " << e.what() << std::endl;
    }
}

```

Output:

```

Starting risky operation
Operation failed: Something went wrong

```

The timing of when to check for exceptions matters. In this example, `check_exception()` is called after `run()` completes. If the coroutine suspended multiple times, you might want to check for exceptions after each resumption.

For generators with iterators, exceptions are typically checked during iteration:

```

iterator& operator++() {
    handle_.resume();
    if (handle_.done()) {
        auto& promise = handle_.promise();
        if (promise.exception) {

```

```

            std::rethrow_exception(promise.exception);
        }
    }
    return *this;
}

```

This ensures that exceptions are propagated to the code iterating over the generator.

Be aware of exception safety during coroutine initialization. If an exception is thrown before the first suspension point (and before `initial_suspend` completes), the exception propagates directly to the caller without going through `unhandled_exception()`. If `initial_suspend()` returns `suspend_always`, the coroutine suspends before any user code runs, avoiding this issue.

You have now learned how to handle exceptions in coroutines. The `unhandled_exception()` method provides a hook for capturing or propagating exceptions, and the stored exception pattern allows callers to receive exceptions even when the coroutine has suspended and resumed.

Step 10 — Practical Patterns and Applications

You have learned the mechanics of C++20 coroutines. Now you will explore practical patterns that demonstrate their power.

Lazy Sequences

Generators excel at producing lazy sequences—sequences where values are computed only when needed. This pattern is useful when working with infinite sequences or when computing values is expensive.

```
Generator<int> infinite_counter()
{
    int i = 0;
    while (true) {
        co_yield i++;
    }
}

Generator<int> primes()
{
    auto is_prime = [](int n) {
        if (n < 2) return false;
        if (n == 2) return true;
        if (n % 2 == 0) return false;
        for (int i = 3; i * i <= n; i += 2) {
            if (n % i == 0) return false;
        }
        return true;
    };

    while (true) {
        int n = 2;
        while (true) {
            if (is_prime(n)) {
                co_yield n;
            }
            ++n;
        }
    }
}
```

```
int main()
{
    int count = 0;
    for (int p : primes()) {
        std::cout << p << " ";
        if (++count >= 10) break;
    }
    std::cout << std::endl;
}
```

Output:

```
2 3 5 7 11 13 17 19 23 29
```

The prime generator tests each number for primality but only computes values as they are requested. An infinite number of primes exist, but the program only computes the first ten.

Transforming Sequences

Generators can transform sequences from other generators, creating a pipeline of operations:

```
Generator<int> take(Generator<int> source, int n)
{
    int count = 0;
    for (int value : source) {
        if (count++ >= n) break;
        co_yield value;
    }
}

Generator<int> filter(Generator<int> source, bool (*predicate)(int))
{
    for (int value : source) {
        if (predicate(value)) {
            co_yield value;
        }
    }
}
```

```
    }
}

Generator<int> transform(Generator<int> source, int (*func)(int))
{
    for (int value : source) {
        co_yield func(value);
    }
}

bool is_even(int n) { return n % 2 == 0; }
int square(int n) { return n * n; }

int main()
{
    // Take first 5 even numbers from range, then square them
    auto pipeline = transform(
        filter(
            take(range(1, 100), 10),
            is_even
        ),
        square
    );

    for (int x : pipeline) {
```

```

        std::cout << x << " ";
    }
    std::cout << std::endl;
}

```

Output:

```
4 16 36 64 100
```

Each generator in the pipeline produces values on demand. The `filter` generator only requests the next value from its source when it needs to produce an output. The `transform` generator only transforms values as they pass through.

Tree Traversal

Ana Lúcia de Moura and Roberto Ierusalimschy, in their influential paper on coroutines, demonstrated tree traversal as a classic use case. With generators, you can traverse a tree structure while maintaining the simple recursive algorithm:

```

struct TreeNode {
    int value;
    TreeNode* left;

```

```

    TreeNode* right;

    TreeNode(int v, TreeNode* l = nullptr, TreeNode* r = nullptr)
        : value(v), left(l), right(r) {}
};

Generator<int> inorder(TreeNode* node)
{
    if (node == nullptr) {
        co_return;
    }

    for (int v : inorder(node->left)) {
        co_yield v;
    }

    co_yield node->value;

    for (int v : inorder(node->right)) {
        co_yield v;
    }
}

int main()
{

```



```

//      4
//      / \
//     2   6
//    / \ / \
//   1  3 5  7

TreeNode n1(1), n3(3), n5(5), n7(7);
TreeNode n2(2, &n1, &n3), n6(6, &n5, &n7);
TreeNode root(4, &n2, &n6);

for (int v : inorder(&root)) {
    std::cout << v << " ";
}
std::cout << std::endl;
}

```

Output:

```
1 2 3 4 5 6 7
```

The recursive structure of the tree traversal matches the recursive structure of the code. Each call to `inorder` creates a new generator that yields values from its subtree. The `co_yield` in the loop forwards those values upward.

Cooperative Multitasking

Coroutines enable cooperative multitasking without threads. Multiple tasks can make progress by voluntarily yielding control:

```

#include <vector>
#include <string>

struct Task {
    struct promise_type {
        Task get_return_object() {
            return Task{std::coroutine_handle<promise_type>::from_promise(*this)};
        }
        std::suspend_always initial_suspend() { return {}; }
        std::suspend_always final_suspend() noexcept { return {}; }
        void return_void() {}
        void unhandled_exception() { std::terminate(); }
    };

    std::coroutine_handle<promise_type> handle;

    Task(std::coroutine_handle<promise_type> h) : handle(h) {}
    ~Task() { if (handle) handle.destroy(); }

    Task(Task&& other) noexcept : handle(other.handle) {

```

```

        other.handle = nullptr;
    }

    bool done() const { return handle.done(); }
    void resume() { handle.resume(); }
};

struct Scheduler {
    std::vector<Task> tasks;

    void add(Task task) {
        tasks.push_back(std::move(task));
    }

    void run() {
        while (!tasks.empty()) {
            for (size_t i = 0; i < tasks.size(); ) {
                tasks[i].resume();
                if (tasks[i].done()) {
                    tasks.erase(tasks.begin() + i);
                } else {
                    ++i;
                }
            }
        }
    }
};

```

```

    }
};

Task worker(std::string name, int iterations)
{
    for (int i = 0; i < iterations; ++i) {
        std::cout << name << " iteration " << i << std::endl;
        co_await std::suspend_always{};
    }
}

int main()
{
    Scheduler scheduler;
    scheduler.add(worker("Alice", 3));
    scheduler.add(worker("Bob", 2));
    scheduler.run();
}

```

Output:

```

Alice iteration 0
Bob iteration 0
Alice iteration 1

```

```
Bob iteration 1
Alice iteration 2
```

The scheduler interleaves the execution of Alice and Bob. Each task runs until it hits `co_await suspend_always{}`, then yields control. The scheduler resumes the next task, achieving cooperative multitasking.

This pattern can be extended with I/O operations. Instead of `suspend_always`, tasks would await I/O completions. A real scheduler would integrate with an event loop, resuming tasks when their I/O operations complete.

You have now seen practical applications of C++20 coroutines. Lazy sequences, sequence transformations, tree traversal, and cooperative multitasking all benefit from coroutines' ability to suspend and resume execution while preserving local state.

Conclusion

In this tutorial, you explored C++20 coroutines from fundamental concepts to practical implementations.

You began by understanding the problem coroutines solve: the fragmentation of logic that occurs when writing asynchronous code with callbacks. Coroutines restore the natural flow of sequential code while maintaining asynchronous behavior.

You learned to recognize coroutines by their keywords: `co_await` for suspension, `co_yield` for producing values, and `co_return` for completion. You discovered that the presence of any of these keywords transforms a function into a coroutine with special runtime behavior.

You examined the mechanics of suspension and resumption, understanding how the coroutine frame preserves local variables on the heap while the coroutine is suspended. The `std::coroutine_handle` provides the mechanism for resuming a suspended coroutine.

You studied the `promise` type, the controller class that customizes coroutine behavior. Its methods—`get_return_object`, `initial_suspend`, `final_suspend`, `yield_value`, `return_void`, `return_value`, and `unhandled_exception`—define how the coroutine initializes, suspends, produces values, completes, and handles errors.

You built a complete generator type that produces sequences of values on demand. The generator manages coroutine lifetime, provides an iterator interface, and propagates exceptions from the coroutine to calling code.

You explored practical patterns: lazy sequences that compute values only when needed, pipelines that transform sequences, tree traversals that maintain recursive structure, and cooperative multitasking that interleaves multiple tasks.

C++20 coroutines provide a foundation for building sophisticated asynchronous systems. The standard library in C++23 and beyond will provide higher-level abstractions built on this foundation. Understanding the mechanisms described in this tutorial will help you use those abstractions effectively and build your own when needed.

For further exploration, consider studying:

- The `std::generator` type introduced in C++23
- Asynchronous I/O frameworks that use coroutines
- The senders and receivers model being developed for C++26
- Real-world applications of coroutines in networking, databases, and user interfaces

Coroutines represent a significant evolution in how C++ programmers can express complex control flow. The ability to write asynchronous code that reads like synchronous code, while maintaining full control over memory and performance, embodies the spirit of C++: abstraction without hidden costs.

Subscribe to My Very Best AI Slop

By Vinnie · Launched 2 months ago

My non-profit: <https://cppalliance.org> My open source: <https://github.com/vinniefalco>

Pledge your support

By subscribing, you agree Substack's [Terms of Use](#), and acknowledge its [Information Collection Notice](#) and [Privacy Policy](#).



10 Likes

Discussion about this post

Comments Restacks



Write a comment...



Rainbow Roxy Feb 5

...

♥ Liked by Vinnie

Hey, great read as always. It makes async look so simple, but what if debugging coroutine state gets tricky in really complex systems?

♡ LIKE (1) 💬 REPLY

🔗 SHARE

1 reply by Vinnie

1 more comment...